

BRF+ in Real Time

- Arundhati Kanungo
Developer Associate, SAP Labs India Pvt. Ltd.

What is BRF+?

BRF+ or BRFplus (Business Rule Framework plus) is a BRMS (Business Rule management system) provided by SAP AG. It comes as a part of SAP NetWeaver ABAP stack, and offers a comprehensive API (application programming interface) and UI (user interface) for business rule definition and processing. It permits you to model rules in an intuitive way, and reuse the existing rules in various applications.

How did BRMS gain prominence?

In 2008, PWC Technology forecasted that business processes, in whatever form they take depend heavily on business rules. Rules drive activities, coordinate data movement and workflow, and provide decision automation in complex situations. For enterprises, BRMSs provide great agility not only because they automate decisions but also because they enable organizations to modify policies or implement new programs quickly.

Also, per Forrester Research (October 2010), Business Rule Processing and policy based SOA was predicted to be one of the top most technology trends from 2011 onwards.

Rank	Technology trend	Impact	"Newness"	Complexity
1	Next-gen BI takes shape, combining real-time access with pervasiveness, agility, and self-service.	Very high	High	High
2	Business rules processing and policy-based SOA move to the mainstream.	Very high	High	High
3	SaaS and cloud-based platforms become standard.	Very	high High	Very high
4	System management enables continued virtualization.	Very high	Moderate	Low

Business rules processing enables rule management by business-side experts. Applications based on business rules processing are moving from niche into mainstream as rules enable the creation of more-responsive, more-personalized experiences based on more-complex conditions.

What challenges do business face without a BRMS like BRF+?

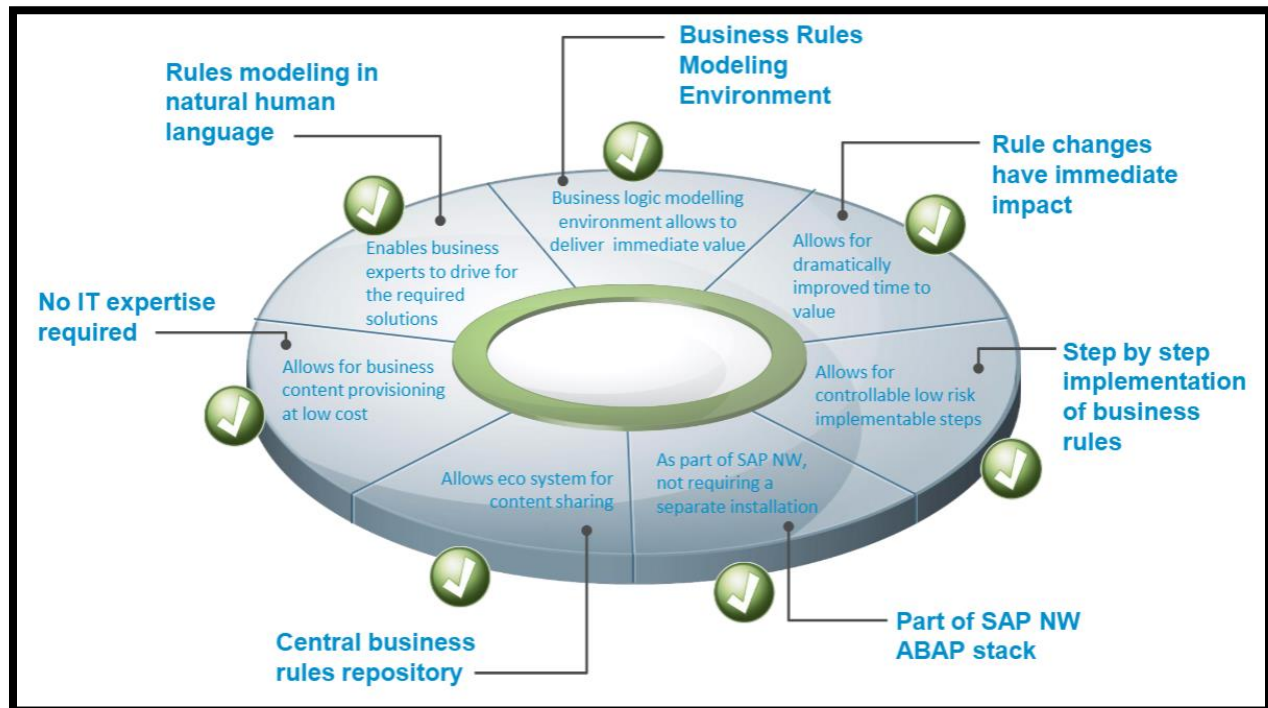
Without BRMS implementation, businesses stand in front of open questions in several areas. Some of them are discussed below.

- **Legal regulations/Security**
 - How can I avoid time-consuming and costly upgrades?
- **Cost reduction, do more with less**
 - How can I avoid the risk and cost of big bang implementations?
- **Increasing importance of externalizing business rules**
 - How can I bring immediate value to our organization?
- **High user expectation**

- How can I deliver immediate innovation?
- **On-demand solutions**
 - How can I enable the business expert to drive IT decisions?
- **Working within a network**
 - How can I deliver business content in a low cost?
 - How can I develop, share and maintain business content?

How BRF+ addresses the above challenges?

BRF+ addresses every challenge in an effective way, with great new features.



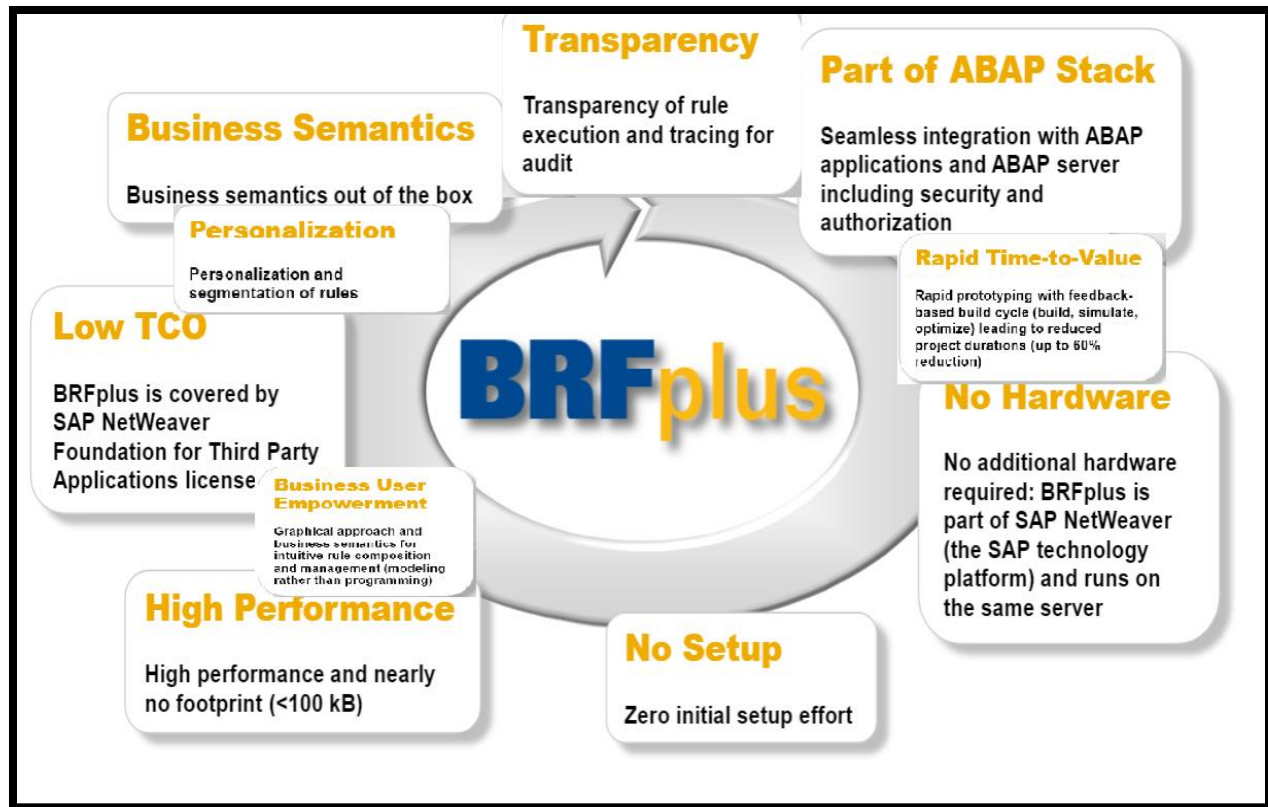
What are the benefits of using BRF+?

Usage of BRF+ has multifarious benefits. Few major advantages include:

- Flexible and agile business systems
- Precise and informed decision making
- Faster turnaround times for changes
- Improved business visibility
- Reduced cost for changes

Why BRF+?

BRF+ is a best of breed rules engine and can do much better in terms of requirements, cost, performance, time, transparency and personalization when compared to other rules engines. Take a quick glimpse here.



Which industries are leveraging BRF+?

Front-runners in the queue are:

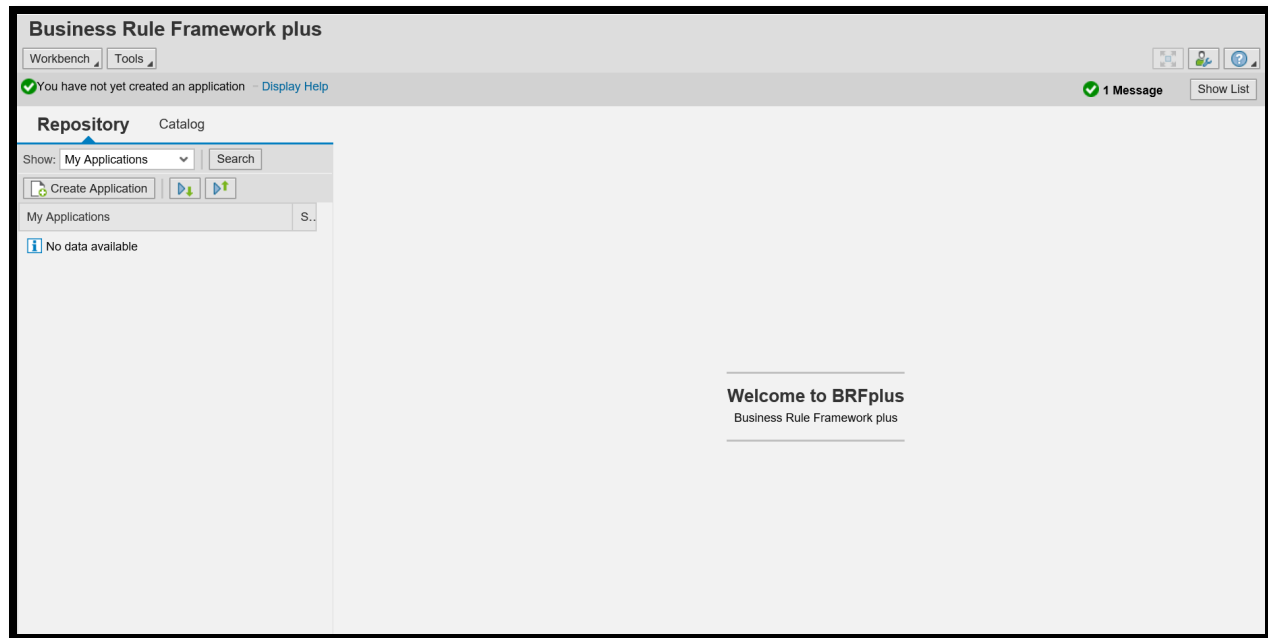
- **Banking:** Relationship based Pricing, Scorecards, Credit Decisioning
- **Education:** Fee Calculations, Course Selections
- **Healthcare:** Claims, Patient Monitoring, Fraud Detection
- **Insurance:** New Products, Claims Settlement, Agent Commissions
- **Public Sector:** Tax Calculations, Customs Duties, Land Regulations, License Fee Calculations
- **Logistics and Shipping:** Parts Management, Duties Calculations, Pricing Calculations

How is a BRF+ application created in real time?

Consider an application to determine the final processing status based on call control, initial processing status and action performed. The determination is done on evaluating rules that might change over time. So, let's start creating a BRF application with the requirements from the above scenario.

Use-case: Determination of final processing status based on call control, initial processing status and action performed

Step 1: Go to BRF+ Workbench with t-code BRF+ or BRFPlus or FDT_Workbench.



Step 2: Create a BRF+ application.

Create Application

General data

Type:

Application

* Name:

STATUS

Short Text:

Status Determination

Text:

Status Determination

Application

Storage Type:

Customizing

Create Local Application:

☒

Development Package:

\$TMP

Software Component:

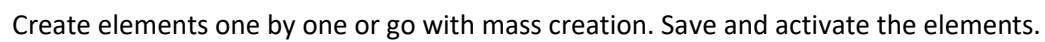
LOCAL

Create And Navigate To Object

Cancel

- System: Transportable objects that can't be changed in system environment
- Customizing: Transportable objects allowing changes
- Master Data: Local objects that can't be transported

Step 3: Create Elements from the path Create -> Data Object -> Element.



Create New Elements

Application: Status Determination

Add Rows

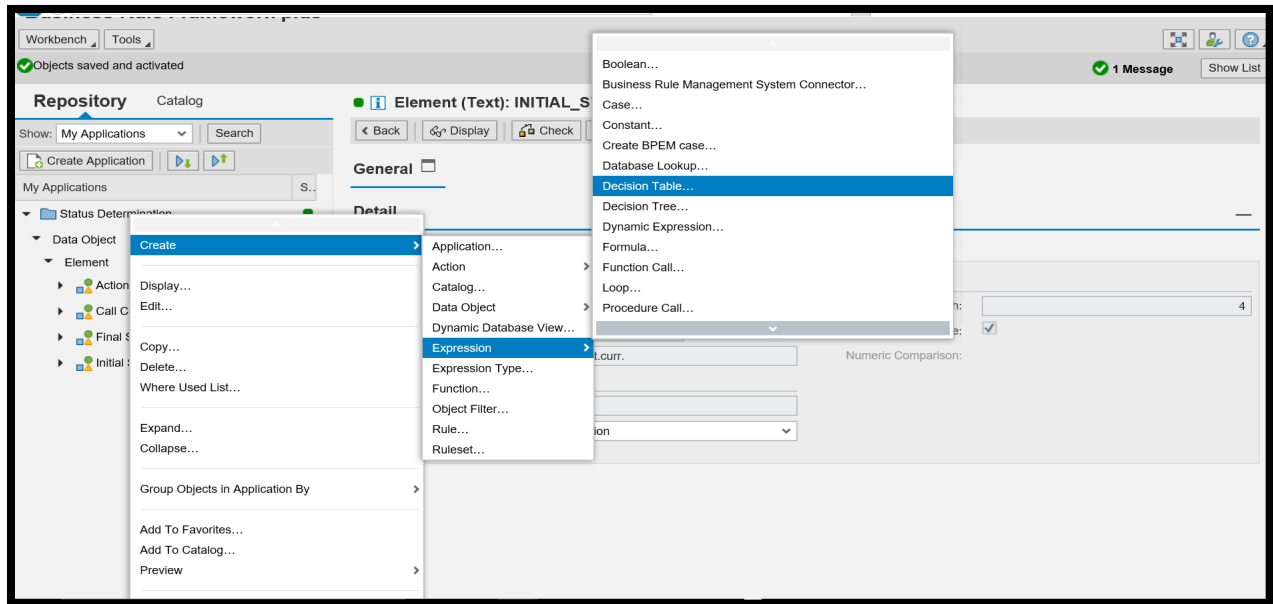
Delete Marked Rows

	Type	Elem...	DDIC...	Elem...	Name	Short Text	Length	Decimals	Only P...	Quantit...	Timep...
	DDI... ▾	<Not... 	JIT_CNTR	Text	all Control	Call Control	4	0	☐		
	DDI... ▾	<Not... 	JIT_INTST	Text	INITIAL S1	Initial Status	4	0	☐		
	DDI... ▾	<Not... 	JIT_AKTIC	Text	ACTION	Action	4	0	☐		
	DDI... ▾	<Not... 	TST_NEW	Text	nal Status	Final Status	4	0	☐		
	Buil... ▾	<Not... 		Text ▾			255	0	☐		

Ok

Cancel

Step 4: Create Decision table from path Create -> Expression -> Decision Table.



Provide Input and Output parameters for the table. Save and Activate the decision table.

Table Settings

Result Data Object

Settings:

- ☐ Return all matches found
- ☒ Return initial value if no match is found
- ☐ Return Exception for partial match

Result Data Object: Final Status

Table Check Settings

Overlap Check Settings: Application Default

Completeness Check Settings: Application Default

List of Columns

Condition Columns

Column Name	Text	Mandatory Input	Column Accessibility
CALL_CONTROL	Call Control	☒	Full Access (Changes Al...)
INITIAL_STATUS	Initial Status	☒	Full Access (Changes Al...)
ACTION	Action	☒	Full Access (Changes Al...)

Result Columns

Column Name	Text	Action Column	Mandatory Input	Column Accessibility
FINAL_STATUS	Final Status	☐	☒	Full Access (Cha...)

OK Cancel

Insert values to the decision table as below.

Table Contents				
Find:	Next	Previous	Table Settings	
Call Control	Initial Status	Action	Final Status	
=AU01	=0010 (JIT Call created (incl. message))	=OAPP (Add Components Group to Outbound Call	0010 (JIT Call created (incl. message))	
=AU01	=0010 (JIT Call created (incl. message))	=OFIN (Complete Outbound Components Group)	0050 (JIT call finished)	
=AU01	=0011 (JIT Call modified)	=OAPP (Add Components Group to Outbound Call	0011 (JIT Call modified)	
=AU01	=0030 (Delivery received/created)	=OCSH (Delete from inbound delivery)	0032 (Delivery deleted/cancelled)	
=AU02	=0010 (JIT Call created (incl. message))	=OAPP (Add Components Group to Outbound Call	0011 (JIT Call modified)	
=AU02	=0011 (JIT Call modified)	=ODLI (Create delivery for JIT Outbound)	0030 (Delivery received/created)	
=AU02	=0030 (Delivery received/created)	=OMOD (Modification of a JIT Outbound Call)	0030 (Delivery received/created)	
=AU02	=0031 (Additional Delivery received/created)	=OGRE (Post Goods Receipt for JIT Outbound)	0040 (Goods Movement posted)	
=AU03	=0030 (Delivery received/created)	=OSDP (Send Outbound Call: Create Message)	0030 (Delivery received/created)	
=AU03	=0040 (Goods Movement posted)	=OCGR (Reverse Goods Receipt)	0042 (Goods Movement cancelled)	
=AU04	=0042 (Goods Movement cancelled)	=OGRE (Post Goods Receipt for JIT Outbound)	0040 (Goods Movement posted)	
=AU04	=0050 (JIT call finished)	=OARC (Archiving Outbound Calls)	0090 (JIT Call for archiving/reorganization)	
=AU11	=1000 (JIT Call created)	=DELI (Create Delivery)	1200 (Delivery created)	
=AU12	=1300 (Goods Movement posted)	=MTCH (Dely. conf.: set to 'matched')	1300 (Goods Movement posted)	
=AU20	=1000 (JIT Call created)	=MODI (Modify sequenced JIT call (total))	1000 (JIT Call created)	

Step 3: Create new rule from the path Create -> Rule.

My Applications | Search | Back | Display | Check | Save | Activate | Delete | More

Create Application | My Applications | S.

General | Detail

My Applications

- Status Definition
- Data Object
 - Create
 - Display...
 - Edit...
 - Copy...
 - Delete...
 - Where Used List...
- Expression
 - Expand...
 - Collapse...
- Decision
 - Group Objects in Application By
 - Add To Favorites...
 - Add To Catalog...
 - Preview
- Pro
- Uses

Application...
Action
Catalog...
Data Object
Dynamic Database View...
Expression
Expression Type...
Function...
Object Filter...
Rule...
Ruleset...

Start Simulation

Initial Status	Action	Final Status
=0010 (JIT Call created (incl. message))	=OAPP (Add Components Group to Outbound Call	0010 (JIT Call created (incl. message))
=0010 (JIT Call created (incl. message))	=OFIN (Complete Outbound Components Group)	0050 (JIT call finished)
=0011 (JIT Call modified)	=OAPP (Add Components Group to Outbound Call	0011 (JIT Call modified)
=0030 (Delivery received/created)	=OCSH (Delete from inbound delivery)	0032 (Delivery deleted/cancelled)
=0010 (JIT Call created (incl. message))	=OAPP (Add Components Group to Outbound Call	0011 (JIT Call modified)
=0011 (JIT Call modified)	=ODLI (Create delivery for JIT Outbound)	0030 (Delivery received/created)
=0030 (Delivery received/created)	=OMOD (Modification of a JIT Outbound Call)	0030 (Delivery received/created)
=0031 (Additional Delivery received/created)	=OGRE (Post Goods Receipt for JIT Outbound)	0040 (Goods Movement posted)
=0030 (Delivery received/created)	=OSDP (Send Outbound Call: Create Message)	0030 (Delivery received/created)
=0040 (Goods Movement posted)	=OCGR (Reverse Goods Receipt)	0042 (Goods Movement cancelled)
=0042 (Goods Movement cancelled)	=OGRE (Post Goods Receipt for JIT Outbound)	0040 (Goods Movement posted)
=0050 (JIT call finished)	=OARC (Archiving Outbound Calls)	0090 (JIT Call for archiving/reorganization)
=1000 (JIT Call created)	=DELI (Create Delivery)	1200 (Delivery created)
=1300 (Goods Movement posted)	=MTCH (Dely. conf.: set to 'matched')	1300 (Goods Movement posted)
=1000 (JIT Call created)	=MODI (Modify sequenced JIT call (total))	1000 (JIT Call created)

Create Rule

Type:

Rule

* Name:

Obtain Status

Short Text:

Obtain Status

Text:

Obtain Status

Application:

STATUS

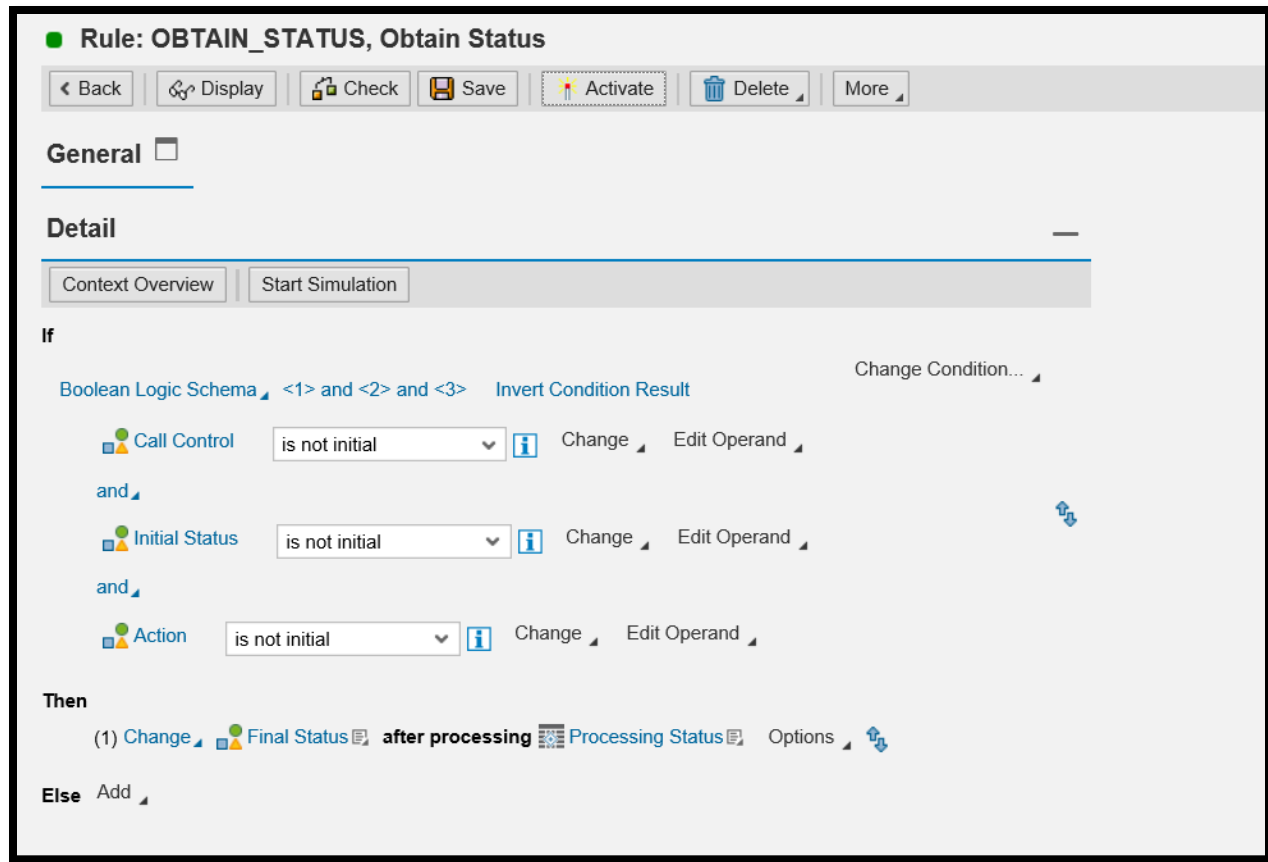
Is Reusable:

☒

Create

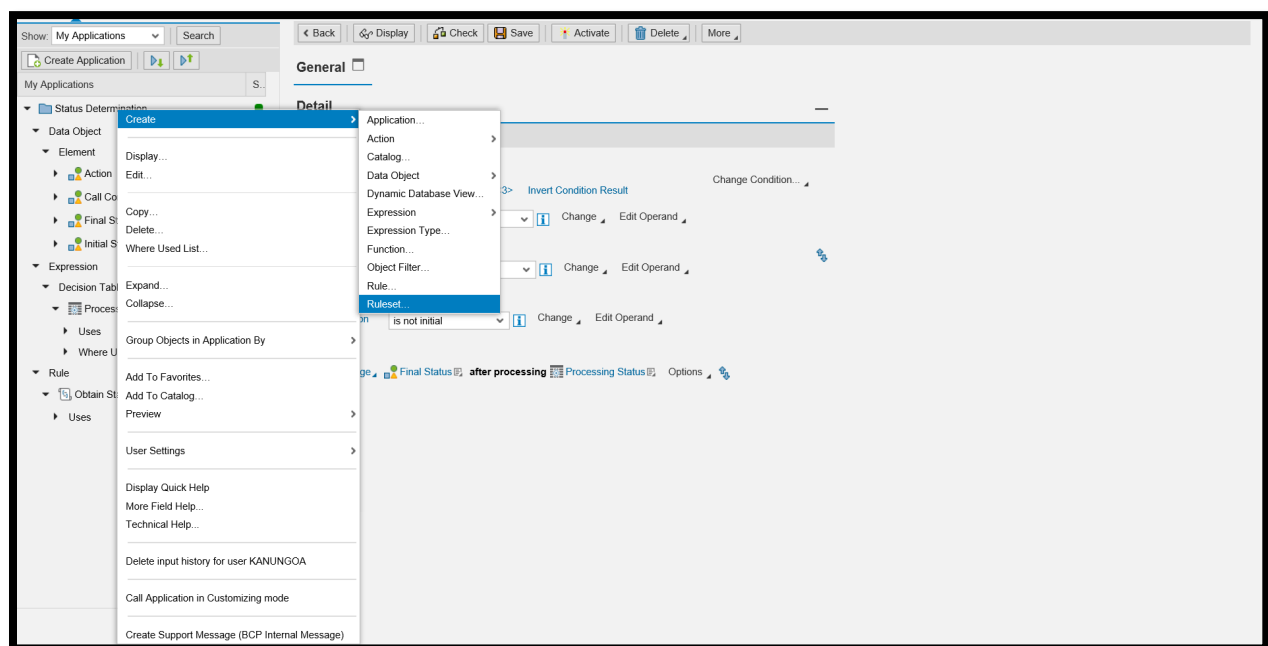
Create And Navigate To Object

Cancel



Save and activate the rule.

Step 4: Create Ruleset from the path Create -> Ruleset.



Create Ruleset

Type: Ruleset

* Name: Obtain Status

Short Text: Obtain Status

Text: Obtain Status

Application: STATUS

Create Create And Navigate To Object Cancel

Ruleset: OBTAIN_STATUS, Obtain Status

< Back Display Check Save Activate Delete More

General

Detail

Show Ruleset Header Context Overview

Rules

Insert Rule Insert Exit Condition

Disable Rule (1) Rule: Obtaining Status - Unlimited Validity Options

Valid From / To: 00:00:00 / 00:00:00

Description: Obtaining Status

Save and activate the ruleset.

Step 5: Create function from path Create -> Function.

Repository Catalog

Show: My Applications Search

Create Application

My Applications

▼ Status Determination

▼ Data Object

▼ Element

▼ Action

▼ Call Control

▼ Final State

▼ Initial State

▼ Expression

▼ Decision Table

▼ Process

▼ Rule

▼ Obtain Status

▼ Ruleset

▼ Obtain Status

Create

Application...

Action

Catalog...

Data Object

Dynamic Database View...

Expression

Expression Type...

Function...

Object Filter...

Rule...

Ruleset...

Display...

Edit...

Copy...

Delete...

Where Used List...

Expand...

Collapse...

Group Objects in Application By

Add To Favorites...

Add To Catalog...

Preview

User Settings

Display Quick Help

More Field Help...

Technical Help...

Delete input history for user KANUNGOA

Call Application in Customizing mode

Ruleset: OBTAIN_STATUS, Obtain Status

< Back Display Check Save Activate Delete More

General

Detail

Show Ruleset Header Context Overview

Rules

Insert Rule Insert Exit Condition

Disable Rule (1) Rule: Obtaining Status - Unlimited Validity Options

Valid From / To: 00:00:00 / 00:00:00

Description: Obtaining Status

Create Function

Type:

Function

* Name:

Status Decision

Short Text:

Status Decision

Text:

Status Decision

Application:

STATUS

Create

Create And Navigate To Object

Cancel

Function: STATUS_DECISION, Status Decision

Back

Display

Check

Save

Activate

Delete

More

General

Detail

Simulation

Debugger

Mode: Event Mode

Signature

Assigned Rulesets

Context

Add Existing Data Object

Add New Data Object

Remove

Component Name	Text	Type
ACTION	Action	Element (Text)
CALL_CONTROL	Call Control	Element (Text)
INITIAL_STATUS	Initial Status	Element (Text)

Result Data Object

Data Object: Final Status

Save and activate the function.

Step 6: Assign the function to your ruleset.

Ruleset: OBTAIN_STATUS, Obtain Status

Back

Display

Check

Save

Activate

Delete

More

General

Detail

Hide Ruleset Header

Context Overview

Enabled: ☒

Function: Status Decision

Precondition: <Not assigned>

Number of Rules: 1

Number of Variables: 0

Priority: 00

Variables

Expressions Which Initialize the Variables

(1) <Not assigned>

(1) <Not assigned>

Rules

Insert Rule

Insert Exit Condition

Disable Rule

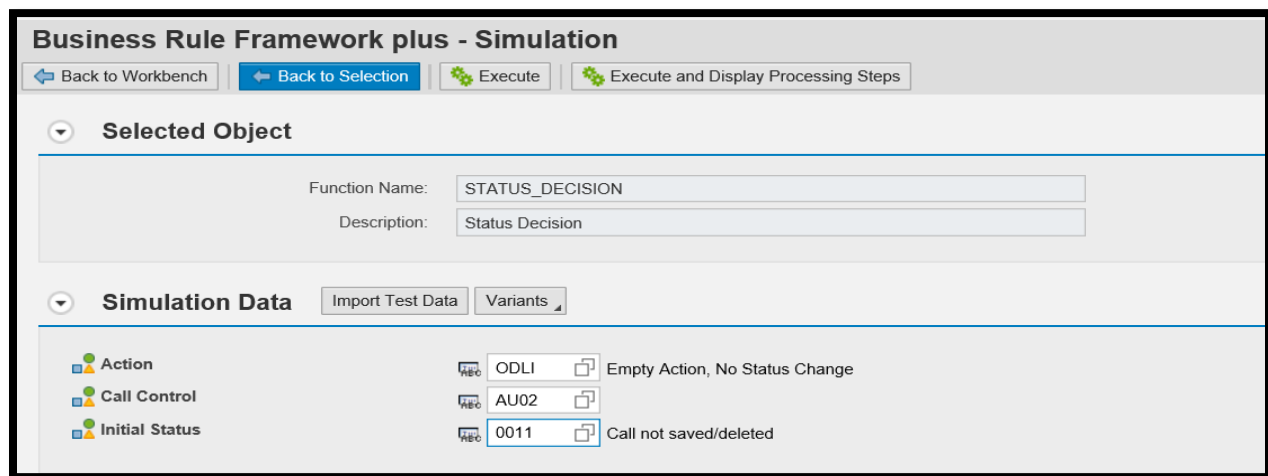
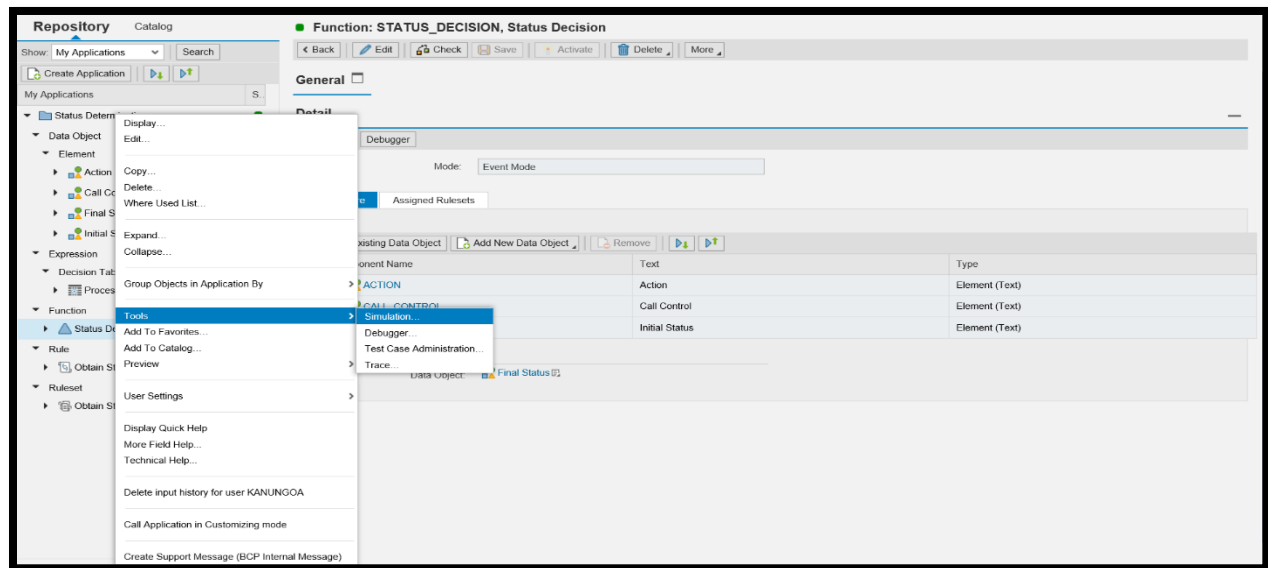
(1) Rule: Obtaining Status - Unlimited Validity

Options

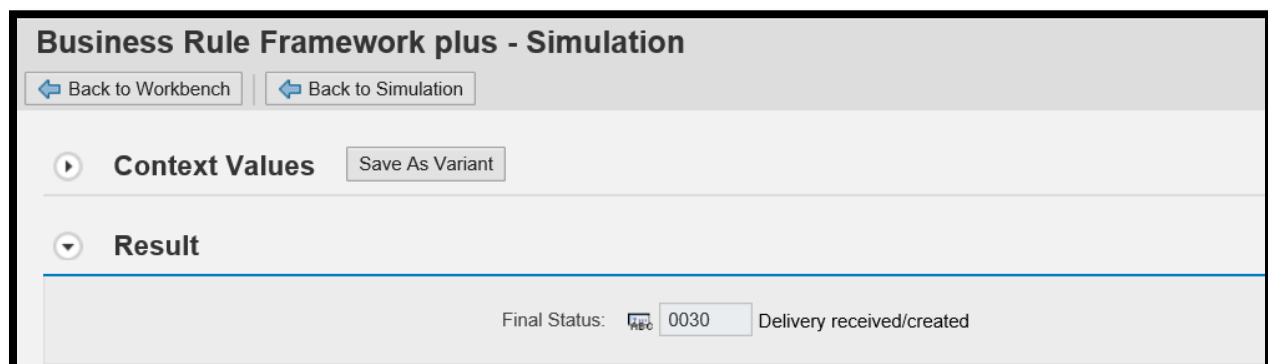
Valid From / To: 1 00:00:00 / 1 00:00:00

Description: Obtaining Status

Step 7: Simulate function.



And here comes the result.



Now you are done with the creation of a simple BRF+ application.

Conclusion:

BRF+ is continued to be used as one of the leading Business Rule Management System due to its flexible, easy and cost-effective implementation, aiming to run business simpler.
